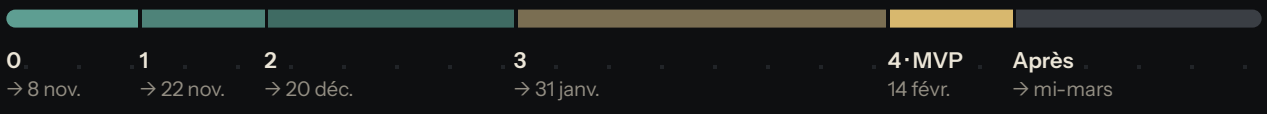
Agent Yattus

FEUILLE DE ROUTE · NOVEMBRE 2026 → MARS 2027

Construire un poste de contrôle pour agents IA.

Les étapes à suivre, de la boucle d'agent jusqu'au MVP. Pour chacune : ce qu'il faut savoir expliquer, et la documentation officielle à lire avant de commencer.



LE PRODUIT

Ce qu'on construit

Agent Yattus te permet de confier une mission à une équipe d'agents IA sans perdre le contrôle. Tu vois tout ce qu'ils font, et rien de sensible ne se passe sans ton accord.

01
Mission
Tu décris le travail en une phrase.

02
Équipe
Des agents aux rôles distincts se passent le relais.

03
Direct
Tu suis chaque étape, son coût et sa durée.

04
Décision
Avant toute action sensible, l'agent te demande ton accord.

05
Trace
Qui a fait quoi, et pourquoi.

Lire ce document

SQUELETTE — Le code existe et ses tests passent, mais il n'est pas encore commité.

PROCHAINE — L'étape sur laquelle on travaille maintenant.

À FAIRE — Prévue, rien n'est encore écrit.

VISION — L'idée est validée, sans date fixée.

- **Essentiel :** à lire avant de commencer l'étape.
- **Approfondissement :** quand l'étape fonctionne.

Une étape est terminée quand tu sais répondre seul aux questions de la colonne « Savoir expliquer ».

Les 7 compétences d'un développeur d'agents

1	La boucle d'agent	phase 0
2	Les outils	phases 0 et 2
3	Les prompts et les rôles	phase 4
4	L'orchestration	phases 1 et 4
5	L'humain dans la boucle	phases 2 et 3
6	L'évaluation	phase 4
7	L'observabilité	phase 2

SOCLE ET BOUCLE D'AGENT

00 Comprendre ce qu'est un agent

Écrire la boucle à la main, sans framework, pour voir ce qu'un agent fait vraiment.

jusqu'au **8 nov.**

0.1

SQUELETTE

Socle .NET 10

Solution `.slnx`, versions des paquets centralisées, avertissements traités comme des erreurs, tests xUnit v3.

SAVOIR EXPLIQUER

- À quoi sert `Directory.Packages.props` ?
- Pourquoi traiter les avertissements comme des erreurs ?

À LIRE

- **Central Package Management**
  `learn.microsoft.com/nuget/consume-packages/central-package-management`
- **Nouveautés de .NET 10**
  `learn.microsoft.com/dotnet/core/whats-new/dotnet-10/overview`
- **Démarrer avec xUnit v3**
  `xunit.net/docs/getting-started/v3/getting-started`

0.2

PROCHAINE

La boucle d'agent écrite à la main

`RunAsync` dans `spikes/AgentLoop` : le modèle demande des outils, le code les exécute et lui renvoie le résultat, jusqu'à la réponse finale. Vérification : 17 tests sur 17.

SAVOIR EXPLIQUER

- Que contiennent un tool_use et un tool_result ?
- Quand la boucle s'arrête-t-elle ?
- Pourquoi fixer un nombre maximal de tours ?
- Que renvoyer au modèle quand un outil échoue ?

À LIRE

- **Building effective agents** (Anthropic)
  `anthropic.com/engineering/building-effective-agents`
- **Tool use, vue d'ensemble**
  `docs.claude.com/en/docs/agents-and-tools/tool-use/overview`
- **Microsoft.Extensions.AI** (`IChatClient`)
  `learn.microsoft.com/dotnet/ai/microsoft-extensions-ai`
- **SDK C# d'Anthropic**
  `github.com/anthropics/anthropic-sdk-csharp`

0.3

À FAIRE

Lecture guidée du squelette

Je pose les questions, tu réponds en lisant le code. Vérification : tu racontes le trajet d'une mission, de `POST /missions` jusqu'à la carte « Terminée ».

SAVOIR EXPLIQUER

- Pourquoi `POST /missions` répond 202 et pas 201 ?
- Qui exécute la mission, et sur quel thread ?
- Comment l'écran apprend-il que la mission est finie ?

À LIRE

- **Minimal APIs**
  `learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/overview`
- **Background services**
  `learn.microsoft.com/aspnet/core/fundamentals/host/hosted-services`
- **System.Threading.Channels**
  `learn.microsoft.com/dotnet/core/extensions/channels`
- **SignalR, introduction**
  `learn.microsoft.com/aspnet/core/signalr/introduction`

0.4

À FAIRE

ADR 0002 : licence du projet

Choisir la licence open source avant de publier le dépôt, et consigner ce choix dans un ADR (Architecture Decision Record).

SAVOIR EXPLIQUER

- MIT, Apache 2.0 ou AGPL : qu'est-ce que chacune autorise ?
- Que contient un ADR, et pourquoi en écrire un ?

À LIRE

- **Choose a License**
  choosealicense.com
- **Architecture Decision Records**
  adr.github.io

0.5

SQUELETTE

Intégration continue

Build et tests sur GitHub Actions. Vérification : le pipeline est vert sur une pull request.

SAVOIR EXPLIQUER

- Pourquoi le projet mobile est-il exclu de la CI ?

À LIRE

- **Build et tests .NET sur GitHub Actions**
  docs.github.com/actions/…/building-and-testing-net
- **Conventional Commits**
  conventionalcommits.org

AGENT FRAMEWORK

01 Passer au framework

Refaire la même mission avec Microsoft Agent Framework, et comprendre ce qu'il fait à notre place.

jusqu'au **22 nov.**

1.1

À FAIRE

La même mission avec Agent Framework

Les mêmes scénarios que le spike de la phase 0 passent, sans boucle écrite à la main.

SAVOIR EXPLIQUER

- Quelle partie de ta boucle le framework remplace-t-il ?
- Qu'est-ce qu'il ajoute : historique, outils, télémétrie ?
- Qu'est-ce que tu perds en contrôle ?

À LIRE

- **Agent Framework, vue d'ensemble**
  learn.microsoft.com/agent-framework/overview/agent-framework-overview
- **Outils d'un agent**
  learn.microsoft.com/agent-framework/user-guide/agents/agent-tools
- **Conversations multi-tours**
  learn.microsoft.com/agent-framework/user-guide/agents/multi-turn-conversation

1.2

À FAIRE

ADR 0003 : architecture du backend

Décrire ce que le squelette a choisi (monolithe modulaire, un schéma de base par module, file d'attente en mémoire) et pourquoi.

SAVOIR EXPLIQUER

- Pourquoi un monolithe modulaire plutôt que des microservices ?
- Qu'est-ce qui est perdu si l'API redémarre en pleine mission ?

À LIRE

- **Architecture Decision Records**
  adr.github.io
- **NetArchTest** (tester les frontières entre modules)
  github.com/BenMorris/NetArchTest

1.3

À FAIRE

ADR 0004 : backend hybride, local ou serveur

Le même binaire d'API tourne en local (SQLite, lancé à côté du desktop) ou sur un serveur (PostgreSQL, authentification). Les clients ne connaissent que l'URL.

SAVOIR EXPLIQUER

- Pourquoi l'API reste-t-elle sur localhost tant qu'il n'y a pas d'authentification ?
- Où vit la clé d'API dans chaque mode ?

À LIRE

- **Secrets en développement**
  `learn.microsoft.com/aspnet/core/security/app-secrets`
- **Kestrel**
  `learn.microsoft.com/aspnet/core/fundamentals/servers/kestrel`

BACKEND

02 Le moteur des missions

Une vraie mission exécutée par un agent, mise en pause pour ton approbation et observable de bout en bout.

jusqu'au **20 déc.**

2.1

SQUELETTE

API, PostgreSQL, exécuteur, audit, hub SignalR

Trois modules (Missions, Agents, Audit) et un test d'architecture qui vérifie leurs frontières.

SAVOIR EXPLIQUER

- Pourquoi un schéma de base par module ?
- Que vérifie un test d'intégration avec WebApplicationFactory ?

À LIRE

- **EF Core**
  `learn.microsoft.com/ef/core`
- **Npgsql pour EF Core**
  `npgsql.org/efcore`
- **Tests d'intégration ASP.NET Core**
  `learn.microsoft.com/aspnet/core/test/integration-tests`

2.2

À FAIRE

Le vrai runner sur Agent Framework

Il remplace le runner provisoire et reprend les outils de lecture du spike. Vérification : des tests avec un client de chat scripté, puis une vraie mission sur Haiku.

SAVOIR EXPLIQUER

- Pourquoi tester avec un client scripté plutôt qu'avec le vrai modèle ?
- Comment empêcher un outil de sortir de son dossier ?

À LIRE

- **Outils d'un agent**
  `learn.microsoft.com/agent-framework/user-guide/agents/agent-tools`
- **Prompt caching**
  `docs.claude.com/en/docs/build-with-claude/prompt-caching`

2.3

À FAIRE

`GET /missions` et migrations

La liste des missions, pour que chaque client ne voie plus seulement les missions qu'il a lancées. Puis des migrations EF Core avant le premier changement de schéma.

SAVOIR EXPLIQUER

- Pourquoi paginer une liste dès le début ?
- Que contient une migration ?

À LIRE

- **Minimal APIs**
  `learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/overview`
- **Migrations EF Core**
  `learn.microsoft.com/ef/core/managing-schemas/migrations`

2.4

À FAIRE

Pause et reprise sur action sensible

L'humain dans la boucle : une mission en pause libère l'exécuteur. Vérification : approuver écrit le fichier, refuser n'écrit rien.

SAVOIR EXPLIQUER

- Où est stocké l'état d'une mission en attente ?
- Quelles actions demandent une approbation, et qui en décide ?

À LIRE

- **Outils soumis à approbation**
  learn.microsoft.com/agent-framework/tutorials/agents/function-tools-approvals
- **Workflows**
  learn.microsoft.com/agent-framework/user-guide/workflows/overview

2.5

À FAIRE

Observabilité avec OpenTelemetry

Vérification : les tokens et la durée de chaque appel au modèle sont visibles.

SAVOIR EXPLIQUER

- Quelle différence entre une trace, un span et une métrique ?
- Que mesurer pour suivre le coût d'une mission ?

À LIRE

- **Observabilité des agents**
  learn.microsoft.com/agent-framework/user-guide/agents/agent-observability
- **OpenTelemetry pour .NET**
  opentelemetry.io/docs/languages/dotnet
- **Conventions sémantiques GenAI**
  opentelemetry.io/docs/specs/semconv/gen-ai

CLIENT DESKTOP

03 Voir et décider

L'application Avalonia : lancer une mission, la suivre en direct et approuver depuis l'écran.

jusqu'au **31 janv.**

3.1

SQUELETTE

Client.Core, coquille, écran Accueil

Le client HTTP et SignalR, le `HomeViewModel`, puis l'écran Accueil conforme à la maquette.

SAVOIR EXPLIQUER

- Pourquoi le ViewModel ne dépend-il d'aucune interface graphique ?
- Pourquoi repasser sur le thread UI quand un événement arrive du hub ?

À LIRE

- **CommunityToolkit.Mvvm**
  learn.microsoft.com/dotnet/communitytoolkit/mvvm
- **Documentation Avalonia**
  docs.avaloniaui.net

3.2

À FAIRE

Écran Mission en direct

Le graphe des agents, la frise des événements et les mises à jour en temps réel par SignalR.

SAVOIR EXPLIQUER

- Que se passe-t-il à l'écran si la connexion au hub tombe ?

À LIRE

- **SignalR, introduction**
  learn.microsoft.com/aspnet/core/signalr/introduction

3.3

À FAIRE

Carte d'approbation

L'agent propose une action, tu vois exactement ce qu'il va faire, puis tu approuves ou tu refuses.

SAVOIR EXPLIQUER

- Quelle information te faut-il pour décider en 5 secondes ?
- Que se passe-t-il si deux clients répondent en même temps ?

À LIRE

- **Outils soumis à approbation**
  learn.microsoft.com/agent-framework/tutorials/agents/function-tools-approvals

3.4

À FAIRE

Écrans Agents et Rapport

La fiche de chaque agent (rôle, outils, modèle, autonomie) et le rapport final d'une mission.

SAVOIR EXPLIQUER

- Quels réglages définissent un « type d'agent » ?

À LIRE

- **Documentation Avalonia**
  docs.avaloniaui.net

MVP

04 Une mission à deux agents

L'Analyste lit le code, le Rédacteur écrit `REPORT.md`, et toi, tu approuves.

jusqu'au **14 févr.**

4.1

À FAIRE

La mission de revue à deux agents

Les prompts des deux rôles et le passage de relais de l'un à l'autre. L'écriture de `REPORT.md` est soumise à ton approbation.

SAVOIR EXPLIQUER

- Que transmet l'Analyste au Rédacteur, et sous quelle forme ?
- Pourquoi deux agents plutôt qu'un seul avec deux consignes ?
- Qu'est-ce qu'un bon prompt de rôle ?

À LIRE

- **Orchestrations multi-agents**
  learn.microsoft.com/agent-framework/user-guide/workflows/orchestrations/overview
- **Prompt engineering**
  docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- **Context engineering for AI agents**
  anthropic.com/engineering/effective-context-engineering-for-ai-agents

4.2

À FAIRE

Évaluation : coût par mission réussie, par modèle

Une série fixe de missions de test, rejouée avec Haiku puis avec Sonnet. On compare ce que coûte une mission réussie, pas le prix au token.

SAVOIR EXPLIQUER

- Qu'est-ce qu'une mission « réussie », et qui la juge ?
- Pourquoi le modèle le moins cher au token n'est-il pas forcément le moins cher par mission ?

À LIRE

- **Définir ses critères et ses tests**
  docs.claude.com/en/docs/test-and-evaluate/develop-tests
- **Prompt caching**
  docs.claude.com/en/docs/build-with-claude/prompt-caching

4.3

À FAIRE

Sécurité, README et démo

Une relecture des risques propres aux applications à base de LLM, puis une démo de bout en bout, approbation comprise.

SAVOIR EXPLIQUER

- Qu'est-ce qu'une injection de prompt passant par un fichier lu par un agent ?

À LIRE

- **OWASP Top 10 pour les applications LLM**
  genai.owasp.org/llm-top-10

APRÈS LE MVP

05 Ouvrir l'outil

Le mobile, le serveur et des agents qui progressent.

jusqu'à **mi-mars**

5.1

SQUELETTE

Mobile : écrans Mission et Décision

L'accueil MAUI existe déjà pour Android et iOS. Il reste à suivre une mission et à l'approuver depuis le téléphone.

SAVOIR EXPLIQUER

- Qu'est-ce que le mobile partage avec le desktop, et qu'est-ce qu'il ne partage pas ?

À LIRE

- **Documentation .NET MAUI**
  learn.microsoft.com/dotnet/maui

5.2

À FAIRE

Authentification, déploiement serveur, sidecar local

L'API sort de localhost seulement une fois l'authentification en place. Le desktop peut aussi lancer son propre backend local.

SAVOIR EXPLIQUER

- Comment un client prouve-t-il qui il est à l'API ?

À LIRE

- **Authentification ASP.NET Core**
  learn.microsoft.com/aspnet/core/security/authentication
- **Docker Compose**
  docs.docker.com/compose

5.3

À FAIRE

MCP et RAG

Brancher des outils externes par MCP (Model Context Protocol), et répondre à partir de tes propres documents (RAG avec pgvector). Un plan pour chacun.

SAVOIR EXPLIQUER

- Quelle différence entre un outil interne et un serveur MCP ?
- Quand le RAG vaut-il mieux qu'un long contexte ?

À LIRE

- **Model Context Protocol**
  modelcontextprotocol.io
- **pgvector**
  github.com/pgvector/pgvector

VISION

Des agents qui progressent

Le modèle n'apprend pas en travaillant. Ce qui s'améliore, c'est ce qu'on lui donne. Et chaque progrès se mesure.

Compétences
Des fiches d'expertise que l'agent charge quand la mission en a besoin.

Mémoire validée
Les leçons de chaque mission, retenues seulement après ton accord.

Graphe de progression
Réussite, approbations et coût, mesurés sur une évaluation fixe, version après version.

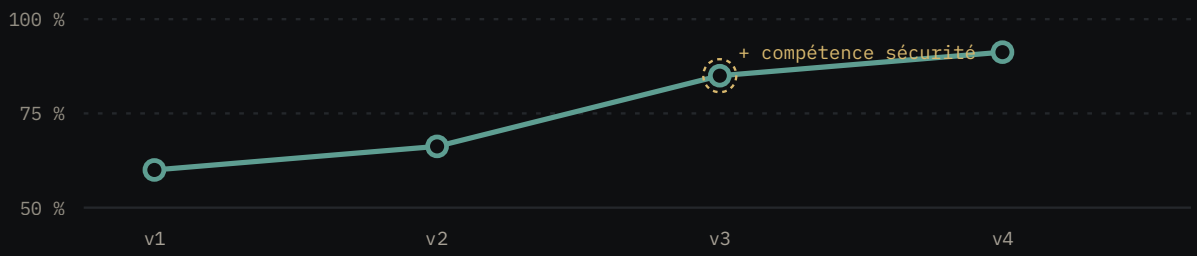


Illustration : taux de réussite fictif sur une évaluation fixe.

À lire : **Agent Skills** `docs.claude.com/en/docs/agents-and-tools/agent-skills/overview`

INDEX

La bibliothèque

Toutes les lectures, classées par thème. Les liens ont été vérifiés le 3 novembre 2026.

AGENTS : LES PRINCIPES

- Building effective agents
  `anthropic.com/engineering/building-effective-agents`
- Context engineering for AI agents
  `anthropic.com/engineering/effective-context-engineering-for-ai-agents`
- Tool use
  `docs.claude.com/en/docs/agents-and-tools/tool-use/overview`
- Prompt engineering
  `docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview`
- Agent Skills
  `docs.claude.com/en/docs/agents-and-tools/agent-skills/overview`
- Prompt caching
  `docs.claude.com/en/docs/build-with-claude/prompt-caching`

ÉVALUATION ET SÉCURITÉ

- Définir ses critères et ses tests
  `docs.claude.com/en/docs/test-and-evaluate/develop-tests`
- OWASP Top 10 pour les applications LLM
  `genai.owasp.org/llm-top-10`

.NET ET IA

- Microsoft.Extensions.AI
  `learn.microsoft.com/dotnet/ai/microsoft-extensions-ai`
- Agent Framework, vue d'ensemble
  `learn.microsoft.com/agent-framework/overview/agent-framework-overview`
- Outils d'un agent
  `learn.microsoft.com/agent-framework/user-guide/agents/agent-tools`
- Outils soumis à approbation
  `learn.microsoft.com/agent-framework/tutorials/agents/function-tools-approvals`
- Orchestrations
  `learn.microsoft.com/agent-framework/user-guide/workflows/orchestrations/overview`
- Observabilité des agents
  `learn.microsoft.com/agent-framework/user-guide/agents/agent-observability`
- Workflows
  `learn.microsoft.com/agent-framework/user-guide/workflows/overview`
- SDK C# d'Anthropic
  `github.com/anthropics/anthropic-sdk-csharp`

BACKEND ASP.NET CORE

- Minimal APIs
  learn.microsoft.com/aspnet/core/fundamentals/minimal-apis/overview
- Background services
  learn.microsoft.com/aspnet/core/fundamentals/host/hosted-services
- Channels
  learn.microsoft.com/dotnet/core/extensions/channels
- SignalR
  learn.microsoft.com/aspnet/core/signalr/introduction
- EF Core et migrations
  learn.microsoft.com/ef/core
- Npgsql pour EF Core
  npgsql.org/efcore
- Authentification
  learn.microsoft.com/aspnet/core/security/authentication
- Secrets en développement
  learn.microsoft.com/aspnet/core/security/app-secrets
- Tests d'intégration
  learn.microsoft.com/aspnet/core/test/integration-tests

CLIENTS

- CommunityToolkit.Mvvm
  learn.microsoft.com/dotnet/communitytoolkit/mvvm
- Avalonia
  docs.avaloniaui.net
- .NET MAUI
  learn.microsoft.com/dotnet/maui

OBSERVABILITÉ

- OpenTelemetry pour .NET
  opentelemetry.io/docs/languages/dotnet
- Conventions sémantiques GenAI
  opentelemetry.io/docs/specs/semconv/gen-ai

SOCLE, CI ET LIVRAISON

- Central Package Management
  learn.microsoft.com/nuget/consume-packages/central-package-management
- GitHub Actions pour .NET
  docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-net
- Nouveautés de .NET 10
  learn.microsoft.com/dotnet/core/whats-new/dotnet-10/overview
- xUnit v3
  xunit.net/docs/getting-started/v3/getting-started
- Docker Compose
  docs.docker.com/compose
- Conventional Commits
  conventionalcommits.org

DÉCISIONS ET OUVERTURE

- Architecture Decision Records
  adr.github.io
- Choose a License
  choosealicense.com
- NetArchTest
  github.com/BenMorris/NetArchTest
- Model Context Protocol
  modelcontextprotocol.io
- pgvector
  github.com/pgvector/pgvector

> Qui écrit le code à chaque étape dépendra du rôle que tu choisis : piloter les agents ou surtout obtenir le produit. Les lectures et les questions « Savoir expliquer » restent les mêmes dans les deux cas : elles te permettent de juger ce que les agents produisent.